# Evaluating Durable Functions for Stateful Serverless Computing

Julio Sigüenza Pacheco

## Abstract

Serverless platforms offer scalability and pay-per-use economics, but they remain poorly suited for stateful and long-running applications. To address this limitation, AWS recently introduced durable functions for AWS Lambda, enabling stateful execution through checkpoint-and-replay mechanisms.

This paper presents a practical investigation of AWS Lambda durable functions through progressively complex applications. We first study a stateful counter to evaluate ordering guarantees, fault tolerance, and deterministic replay. We then explore a video encoding pipeline that orchestrates parallel tasks and manages workflow state across failures. For both cases, we compare durable execution against traditional serverless designs based on explicit external state management.

Our evaluation, conducted through local execution and controlled simulations, shows that durable functions significantly reduce coordination complexity while preserving correctness and fault tolerance. Finally, we analyze durable functions through the lens of the actor model, highlighting their alignment with actor-like semantics in serverless environments and discussing their limitations. These findings position durable functions as a promising abstraction for stateful serverless systems.

## 1. Introduction

Serverless computing promises elastic scale and pay-per-use billing, but its stateless nature complicates long-running workflows. AWS Lambda Durable Functions (preview, December 2025) introduce checkpoint-and-replay primitives that keep state without persistent compute. We evaluate the SDK through a counter entity and a video-encoding pipeline, and position the service within the actor-model landscape. Our contributions are:

- Practical assessment of the SDK (counter + video pipeline).
- Cost/latency comparison with traditional Lambda + DynamoDB.
- Conceptual analysis of durable functions as actors.

## 1 Background and Related Work

### 1.1 Serverless Computing and the State Challenge

Serverless computing has become a dominant paradigm for deploying cloud applications due to its elasticity, fine-grained billing model, and reduced operational overhead. Platforms such as AWS Lambda allow developers to execute short-lived functions without managing servers, automatically scaling based on demand.

Despite these advantages, traditional serverless platforms are inherently stateless. Each function invocation executes in isolation, with no built-in mechanism for preserving execution state across invocations. Applications that require coordination, long-running workflows, or fault-tolerant state transitions must therefore rely on external storage systems such as databases or object stores. This approach increases system complexity, introduces additional latency, and often leads to duplicated or error-prone coordination logic.

### 1.2 Durable Execution and Workflow Orchestration

To address the limitations of stateless serverless execution, several systems have introduced the concept of *durable execution*. In this model, the execution progress of a workflow is persistently recorded, allowing it to be resumed, replayed, or retried transparently after failures.

A prominent example is Azure Durable Functions, which implements deterministic workflow orchestration through event sourcing and replay. Developers write workflows as ordinary functions, while the platform records logical execution steps and replays them during recovery.

More recently, AWS has introduced Durable Execution for AWS Lambda, extending similar concepts to the AWS ecosystem. This approach integrates checkpointing and replay directly into the Lambda runtime, enabling long-running workflows and stateful coordination without explicit state management code.

### 1.3 Actor Model and State Encapsulation

The actor model provides a useful abstraction for reasoning about stateful and concurrent systems. In this model, actors encapsulate state and behavior, processing messages sequentially and communicating exclusively through message passing. This design simplifies reasoning about concurrency and fault isolation.

Durable serverless workflows can be interpreted as a constrained realization of the actor model. Each workflow instance corresponds to a logical actor, workflow steps represent message handlers, and persisted checkpoints encode the actor's durable state. This perspective is valuable for understanding how durable execution systems provide consistency, failure recovery, and isolation while maintaining a functional programming interface.

### 1.4 Related Systems and Prior Work

Several prior systems have explored serverless orchestration for data-intensive workloads. ExCamera demonstrated large-scale video encoding using serverless functions coordinated through external storage and custom control logic. While effective, such approaches require explicit handling of retries, coordination, and state persistence.

Durable execution platforms aim to reduce this burden by embedding fault tolerance and state management directly into the execution model. Rather than exposing low-level coordination primitives, these systems provide structured workflow abstractions that simplify development and improve correctness.

### 1.5 Positioning of This Work

This work builds upon existing research in serverless workflows and durable execution but differs in its emphasis on comparative evaluation. We analyze durable execution against traditional Lambda-based designs with explicit state management using concrete, reproducible workloads.

Specifically, we evaluate a stateful counter and a video processing pipeline implemented using both durable execution primitives and baseline architectures relying on explicit state coordination. The goal is not to propose a new system, but to empirically and conceptually assess the trade-offs introduced by durable execution in modern serverless platforms.

## 2 Architecture and Durable Execution Primitives

This section describes the execution model and programming primitives used in our study. We focus on the durable execution abstraction provided by AWS Lambda and explain how checkpointing and replay are exposed to developers through a step-based programming model.

### 2.1 Durable Workflow Model

A durable workflow is expressed as a deterministic function composed of discrete execution steps. Each workflow instance represents a logical execution context whose progress is transparently persisted by the runtime. The workflow may span multiple physical executions while preserving logical continuity.

The runtime records the inputs and outputs of each step, enabling the workflow to be resumed or replayed after failures without re-executing completed steps. From the developer perspective, the workflow is written as a standard function, while durability guarantees are provided by the platform.

### 2.2 Checkpointing and Replay Semantics

Checkpointing occurs at step boundaries. When a step completes successfully, its result is persisted. In the event of a failure, the runtime reconstructs the workflow state by replaying the execution from the beginning, substituting stored results for completed steps.

Replay is deterministic: side effects must be isolated within steps, and non-deterministic operations are prohibited outside step boundaries. This constraint ensures that replayed executions produce identical control flow and state transitions.

### 2.3 Programming Interface

In our implementation, workflows interact with the durable runtime through a `DurableContext`. This context exposes a `step()` method, which is used to invoke durable steps.

Each step is defined as a pure function decorated with a durable execution annotation. The workflow logic remains sequential and readable, while durability concerns are handled transparently by the runtime.

### 2.4 Example: Stateful Counter Workflow

The counter workflow implemented in Phase 1 demonstrates the basic use of durable steps. Initialization, state updates, and reads are all expressed as individual steps. This design allows the workflow to tolerate failures and retries without manual state recovery logic.

Compared to a baseline implementation using explicit storage, the durable version eliminates the need for explicit read-modify-write cycles and coordination logic, resulting in simpler and more maintainable code.

### 2.5 Example: Video Processing Pipeline

The video processing pipeline implemented in Phase 2 extends the same execution model to a multi-step, fan-out/fan-in workload. The workflow validates metadata, partitions the input video into chunks, encodes each chunk independently, and finally assembles the output video.

Each encoding task is represented as a durable step. Failures during chunk processing trigger automatic retries without affecting completed chunks. The final assembly step executes only after all encoding steps have successfully completed.

This architecture highlights how durable execution simplifies coordination and failure handling in complex workflows without introducing explicit orchestration state.

—

## 3 Evaluation

This section evaluates AWS Lambda durable functions through two progressively complex case studies. We compare durable execution against traditional serverless designs that rely on explicit external state management. Our evaluation focuses on correctness, fault tolerance, code complexity, and operational overhead.

### 3.1 Phase 1: Stateful Counter

The first case study implements a stateful counter supporting increment, decrement, and read operations. Although functionally simple, the counter provides a minimal yet representative workload for evaluating durable execution semantics.

*Correctness and Ordering.* The durable counter preserves strict operation ordering. Concurrent or repeated invocations are serialized by the durable execution context, ensuring deterministic state transitions. Local tests confirm that replays do not re-execute completed steps and that state remains consistent under retries.

*Failure Handling.* We injected transient failures using a deliberately flaky step. When failures occurred, the runtime automatically replayed the workflow from the last successful checkpoint. Completed steps were not re-executed, demonstrating correct checkpoint-and-replay behavior.

*Baseline Comparison.* In the baseline implementation, the counter state is stored explicitly using an external key-value store. This design requires manual coordination, explicit read-modify-write logic, and careful handling of race conditions. The durable implementation eliminates this complexity while preserving equivalent functionality.

### 3.2 Phase 2: Video Processing Pipeline

The second case study evaluates a video encoding pipeline inspired by prior serverless video processing systems. The workflow consists of validation, chunking, parallel encoding, and final assembly.

*Durable Pipeline Execution.* In the durable implementation, each pipeline stage is expressed as a durable step. Video chunks are encoded independently, and failures in individual chunks trigger automatic retries without affecting completed work. The final assembly step executes only after all encoding steps have completed successfully.

Local execution confirms that the workflow tolerates failures during chunk processing and resumes execution without manual intervention or global restarts.

*Explicit-State Baseline.* The baseline pipeline uses explicit job and chunk state stored in an external store. Each stage updates job metadata and chunk status manually. This approach requires significantly more coordination logic, including polling, status tracking, and failure propagation.

Instrumentation of the baseline reveals a large number of store operations, even for a small input video. These operations increase code complexity and introduce additional latency and cost.

### 3.3 Code Complexity and Maintainability

Across both case studies, durable execution reduces code complexity by removing explicit state management and coordination logic. Durable workflows are expressed in a sequential, readable manner, while fault tolerance is delegated to the runtime.

The baseline implementations require additional abstractions for state storage, retries, and consistency, making them harder to reason about and maintain.

### 3.4 Cost and Performance Considerations

While our evaluation is primarily qualitative, we simulated cost behavior using a simplified model. Durable execution avoids billing for idle wait time by suspending workflows between steps. In contrast, baseline implementations incur repeated compute and storage costs due to polling and state updates.

Figure 1 illustrates a simulated cost comparison between durable execution and explicit-state designs for increasing workflow complexity.

### 3.5 Summary

The evaluation demonstrates that durable functions provide strong correctness and fault-tolerance guarantees while significantly simplifying application logic. For stateful and long-running workflows, durable execution reduces developer burden and improves reliability compared to traditional serverless approaches.

—

## 4 Durable Functions and the Actor Model

The actor model is a foundational abstraction for building concurrent and distributed systems. Actors encapsulate state, process messages sequentially, and interact exclusively through asynchronous communication. This section analyzes how AWS Lambda durable functions relate to the actor model and evaluates the extent to which they enable actor-like programming in a serverless environment.

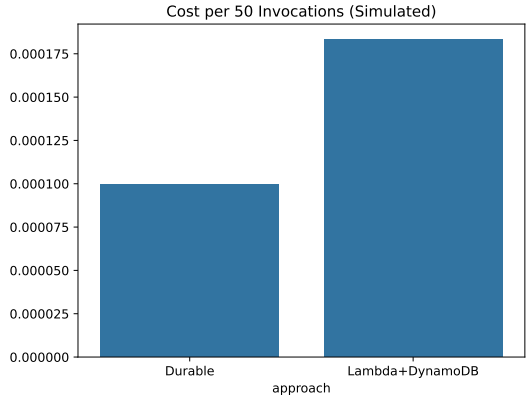


**Figure 1: Simulated cost comparison between durable execution and explicit state management.**

### 4.1 Actor Model Fundamentals

In the classical actor model, each actor: (i) owns private state, (ii) processes messages one at a time, (iii) can create new actors, and (iv) communicates exclusively through message passing. These properties simplify reasoning about concurrency and fault tolerance.

However, traditional serverless platforms struggle to support actors due to stateless execution, lack of message ordering guarantees, and the absence of durable identities.

### 4.2 Durable Functions as Actor-Like Abstractions

AWS Lambda durable functions partially bridge this gap by introducing durable execution contexts with checkpoint-and-replay semantics. A durable execution instance can be identified by a stable execution name, which serves a role similar to an actor identity.

Within a durable execution: state is persisted implicitly through checkpoints, steps are executed sequentially, and failures trigger deterministic replays rather than ad-hoc retries.

These properties closely align with actor semantics. In particular, the durable counter entity implemented in Phase 1 functions as a classical actor: it maintains internal state, processes operations in order, and guarantees atomic updates.

### 4.3 Mapping Durable Primitives to Actor Concepts

Table 1 summarizes the conceptual correspondence between actor model primitives and AWS durable execution mechanisms.

Unlike traditional actor systems, durable functions do not expose explicit mailboxes or message queues. Instead, message handling is implicit in the invocation and replay mechanism.

| Actor Model Concept | Durable Execution Mechanism |
|---|---|
| Actor identity | Execution name |
| Message | Durable execution invocation |
| Mailbox ordering | Sequential step execution |
| Actor state | Checkpointed local variables |
| Failure recovery | Deterministic replay |

**Table 1: Conceptual mapping between the actor model and durable execution.**

## 4.4 Comparison with Related Systems

*Azure Durable Functions.* Azure Durable Functions provide Durable Entities, which explicitly expose actor-like abstractions. AWS durable functions offer similar semantics but emphasize general durable workflows rather than dedicated actor APIs. This design choice reflects AWS's preference for composable primitives over specialized abstractions.

*Traditional Actor Frameworks.* Systems such as Akka or Orleans provide richer actor semantics, including dynamic actor creation and supervision hierarchies. However, these systems require persistent infrastructure and incur idle resource costs. Durable functions trade expressiveness for serverless scalability and pay-per-use economics.

*Serverless Video Pipelines.* In Phase 2, each video chunk encoder can be interpreted as a short-lived actor, while the pipeline orchestration acts as a supervising coordinator. Durable execution enables this structure without explicit actor runtime management.

## 4.5 Limitations and Open Questions

Despite their alignment with actor principles, durable functions lack certain features of full actor systems, such as explicit actor-to-actor messaging and dynamic topology management. Additionally, the semantics of exactly-once execution depend on the correctness of the underlying replay mechanism and external side effects.

Nevertheless, durable functions represent a significant step toward practical actor-like programming in serverless platforms.

## 4.6 Summary

AWS Lambda durable functions enable a restricted but powerful form of actor-based programming. By combining implicit state management, deterministic execution, and fault tolerance, they allow developers to express actor-like behavior without maintaining dedicated runtime infrastructure. This positions durable functions as a promising abstraction for stateful serverless applications.

# 6. Conclusion and Future Work

We evaluated AWS Lambda Durable Functions through a stateful counter and a video encoding pipeline using local execution and controlled simulations. Our study shows that durable execution significantly reduces coordination complexity compared to traditional Lambda-based designs with explicit external state management, while preserving deterministic execution and strict ordering semantics compatible with the actor model.

Although our cost analysis is based on simulated models rather than production deployments, the results suggest potential economic advantages by avoiding idle compute time and redundant state operations. Future work includes evaluating durable functions on real AWS deployments, exploring multi-region workflows, and investigating integrations with AI-driven applications that require long-lived, stateful interactions.

# References